# Waterfall IR Design

Research notes + IR reference

BMA Standard Formulas

May 01, 2026

# Waterfall IR Design — research notes + IR reference

**Status:** Initial design synthesis from 13 prospectus deep reads across RMBS and auto ABS, plus full IR element reference and a worked example. Not a final spec for IR change yet. Part 1 documents what real prospectus language looks like; Part 2 documents the existing IR schema and how to write a deal in it both reusably and readably.

**Sample size (13 deals + FNR 2006-018 fixture):**

| Asset class | Deal | Key features |
|---|---|---|
| Agency MBS REMIC | FNR 2006-018 (fixture) | 2 collateral groups, PAC + Z + Support, face-weighted support split |
| Agency MBS REMIC | FNR 2016-104 | 9 collateral groups, mix of pass-through, sequential, accretion-directed, PAC, face-weighted splits |
| Agency MBS REMIC | FNR 2019-17 | 7 collateral groups, **nested face-weighted splits**, **named Aggregate Group** abstraction |
| Agency Multifamily REMIC | FNMA 2024-M2 | Multifamily; structurally similar to single-family REMICs |
| Agency Synthetic CRT | CAS 2024-R05, CAS 2024-R06 | Connecticut Avenue Securities — synthetic credit risk transfer; |

| Asset class | Deal | Key features |
| --- | --- | --- |
| | | reference pool of FNMA-acquired loans, M-1 / M-2 / B-1 / B-2 notes, **reverse-seniority bond writedowns** on reference-pool losses (sometimes with later writeup), pro-rata principal pre-stepdown / sequential post-stepdown |
| Agency MBS REMIC | Ginnie Mae 2025-203 | **Confirms the FNR PAC + Z + Support pattern is industry-standard**; “Aggregate Scheduled Principal Balance” same abstraction as Fannie’s “Aggregate Group Planned Balance” |
| Agency MBS REMIC | Ginnie Mae 2025-009 (HECM) | Reverse-mortgage REMIC; **Deferred Interest Amount** (catch-up rule type not in current IR) |
| Agency MBS REMIC | Ginnie Mae 2024-115 (Multifamily) | Multifamily-specific: **trustee fee % of Principal Distribution Amount** before cascade |
| Freddie Mac REMIC general OC | (offering circular) | **Single-Tier vs Double-Tier Series**: REMIC-inside-REMIC trust structure (mostly transparent for cashflow IR) |
| Non-Agency RMBS (subprime) | JPMMT 2006 | Single pool, **interest waterfall + principal waterfall** sub-streams, **stepdown date**, **trigger event override**, OC + excess interest, M-1..M-10 mezz with reverse-seniority loss allocation |
| Non-Agency RMBS (Non-QM) | Verus 2024-9 / 2026-4 | **Step-up coupon** at year 5 (time-conditional rate), **LCF (Last Cashflow) class**, mixed pay (senior pro rata, mezz/sub sequential), Class XS for excess spread |
| Prime Auto ABS | Ford Credit Auto Owner Trust 2024-C | Single pool, **interleaved I/P**, named priority principal amounts, target OC build, reserve replenishment, capped trustee fee with overflow |
| Prime Auto ABS | Toyota Auto Receivables 2024-A | Same shape as Ford Credit. **Yield Supplement Overcollateralization Amount** for sub-WAC loan adjustment. |

| Asset class | Deal | Key features |
|---|---|---|
| | | Confirms prime auto = same grammar across sponsors. |
| Auto Lease ABS | Toyota Lexus Owner Trust 2024-A (TLOT) | Lease-specific simpler waterfall (8 steps): pro-rata interest across all classes, single combined priority principal amount, “Securitization Value” valuation. Otherwise same grammar. |
| Subprime Auto ABS | Santander Drive 2024-2 (SDART) | **Same shape as Ford / Toyota prime auto.** Parametric differences (4 mezz classes vs 2-3, 5 named allocations vs 3, \$300K trustee cap vs \$375K). |
| Subprime Auto ABS | Westlake 2024-1 (WLAKE) | 8-class structure (A-1, A-2, A-3 + B, C, D, E). Same waterfall shape as SDART. Confirms subprime auto pattern is consistent across sponsors. |

**Key cross-asset finding:** every asset class reduces to a small set of structural primitives. Prime auto + subprime auto + auto lease all share one grammar. Agency MBS REMICs across Fannie / Freddie / Ginnie share the same PAC + Z + Support pattern. The existing IR captures most of these patterns; the major real gap is **named computed distribution amounts** (heavy in non-agency RMBS and auto), and the **authoring practice** of fragmenting waterfall steps into one rule per bond (a fixture / generator issue, not an IR limitation).

**Asset classes still to cover** (not yet in sample): credit card master trusts, CLOs (managed), marketplace consumer (SoFi / LendingClub / Affirm), equipment ABS, aircraft ABS, solar ABS.

---

# Methodology

For each deal: locate the “Distributions” / “Priority of Payments” section verbatim, categorize each numbered step in the prospectus, and note what abstractions the prospectus implicitly assumes (named amounts, computed amounts, named groups).

---

# RMBS — Agency MBS REMIC

### FNR 2006-018 (existing fixture, anchor)

Two collateral groups, Group 1 has PAC + Z + Support classes with a 95.65 / 4.35 face-weighted support split. Group 2 is a sequential cascade with notional IO.

## FNR 2016-104 (Dec 2016)

**9 separate collateral groups** in one trust. Each group has its own mini-waterfall. Sample group structures:

- **Group 1 (Pass-Through):** "Group 1 Principal Distribution Amount to BA until retired."
    - Single rule, single target.
- **Group 4 (Z + Sequential):** Two rules:
    1. "The Z Accrual Amount to A and B, **in that order**, until retired, and **thereafter** to Z."
    2. "The Group 4 Cash Flow Distribution Amount to A, B and Z, **in that order**, until retired."
    - Each rule has *multiple* targets in a sequential list.
- **Group 5 (PAC + Support):** Three numbered phases:
    1. To Aggregate Group **to its Planned Balance**.
    2. To LZ until retired.
    3. To Aggregate Group **to zero**.
    - Phase 1 = PAC schedule cap (`cap_mode=PLANNED`).
    - Phase 2 = pure sequential, no cap.
    - Phase 3 = cleanup (`cap_mode=NONE`).
    - **The whole "PAC + Support" pattern is three numbered rules, not three rules per bond.**
- **Group 6 (Face-Weighted Split):**
    - "The Group 6 Cash Flow Distribution Amount as follows:
        - 67.8420172533% to QA, QV and QZ, in that order, until retired
        - 32.1579827467% to QB, QW and QY, in that order, until retired"
    - Single SPLIT_CASH at the named percentage with two sub-cascades.

## FNR 2019-17 (Mar 2019)

7 collateral groups. Adds two patterns we did not see in 2006-018 or 2016-104:

- **Pro-rata pay** as a top-level option: "Group 1 Principal Distribution Amount to A and FA, **pro rata**, until retired." — payment_style=PRO_RATA on a multi-target rule.
- **Nested face-weighted splits** in Group 7:
    - "Group 7 … Aggregate Group III as follows:
        - 16.6666666667% to MA until retired, **and**
        - 83.3333333333% as follows:
            - first, 44.8738331794% to ME until retired, **and** 55.1261668206% to MG and MB, in that order, until retired,
            - second, to MC until retired."
    - The 83% branch is itself a SPLIT_CASH with two sub-streams, AND that split has two sequential phases ("first" / "second").
    - **Recursive structure**: rules can contain sub-rules.
- **Named "Aggregate Group" abstraction**: "Aggregate Group III consists of the MA, ME, MC, MG and MB Classes … has a principal balance equal to the aggregate principal balance of the Classes included in Aggregate Group III."
    - A bond *bundle* with its own planned balance and internal allocation rules. **Not a separate REMIC class** — a virtual grouping for schedule cap and downstream rules.

## Common patterns across all 3 agency REMIC deals

- One waterfall per collateral group.

- Each waterfall has 1-3 logical steps.
- **Each step is a multi-target rule**, not one rule per bond.
- Order language is verbatim from the prospectus: “in that order, until retired”, “concurrently / pro rata”, “to its Planned Balance”, “to zero”.
- Z-bond accrual is its own pre-step: “Accrual Amount to X until retired, then to Z” (a sequential cascade itself).
- Face-weighted splits are first-class and **can nest**.
- “Aggregate Group” is a named bond bundle with its own schedule.

---

# Agency Synthetic CRT (CAS 2024-R05, CAS 2024-R06)

Connecticut Avenue Securities (CAS) — Fannie Mae’s flagship credit risk transfer program. Structurally distinct from cash REMICs but **fully in scope** for the IR.

## Structure

- **Reference pool**, not a real cash pool. The trust holds no mortgage collateral — it holds Fannie Mae’s payment obligations derived from the *performance* of a reference pool of FNMA-acquired loans. The reference pool’s amortization, prepayments, and credit events drive the bonds.
- **Note classes** (typical CAS structure): M-1, M-2, B-1, B-2 plus unrated B-3H. M-1 is most senior of the issued notes; B-2 is most junior. There is also a hypothetical reference tranche stack (A-H, M-1H, M-2H, B-1H, B-2H, B-3H) used to compute payments but with no actual notes.
- **No reserves, no excess spread, no OC.** Credit enhancement is pure subordination — junior notes absorb losses before senior.

## Cashflow mechanics

Two simple things happen each period:

1. **Interest** — each note accrues coupon (typically SOFR + spread) on its outstanding balance and Fannie Mae pays that coupon monthly. Like a normal floater.
2. **Principal & writedowns** — the reference pool’s scheduled amortization + prepayments are allocated to bonds (pro-rata pre-stepdown, sequential post-stepdown), and any reference-pool credit losses are *written down* against bond balances in reverse seniority (B-2 first, then B-1, M-2, M-1).

## What CRT needs from the IR

CRT is the **simplest waterfall structurally** but the **hardest without** `BondDef.loss_treatment`. Because there is no cash collateral, the bonds’ economic existence IS their balance — and losses are the primary state change. Specifically:

- **Bond writedown semantics** — when LOSS is allocated to a bond, its balance must decrease and future coupon must accrue on the reduced balance. This is `BondDef.loss_treatment = WRITEDOWN` (proposed addition E in this document).
- **Bond writeup semantics** (rare but real) — if Fannie Mae later determines a credit event was overstated, the writedown is reversed. The bond’s balance increases and the bondholder receives a “writeup payment” representing missed coupon. This is `BondDef.writeup_enabled = true`.

- **Stepdown date + performance triggers** — same idea as non-agency RMBS: pre-stepdown is pro-rata, post-stepdown is sequential, but a delinquency trigger reverts to sequential early if collateral underperforms. Expressible with the proposed `WaterfallBranch` (proposed addition C).
- **Reference-pool collateral input** — the IR's existing `Loan` / `DealRunInput` types accept the reference pool's cashflows directly (treated as if they were real); this part works today.

### Match with current IR

| CRT feature | Current IR | Status |
|---|---|---|
| Sequential / pro-rata principal cascade | `PAY_PRINCIPAL` with `payment_style` | ✅ |
| Reverse-seniority loss cascade | `PAY_WRITEDOWN` with reverse `to_targets` | ✅ |
| **Bond balance writedown on loss** | none | ❌ (need `loss_treatment`) |
| Bond writeup on loss reversal | none | ❌ (need `writeup_enabled` + `PAY_WRITEUP`) |
| Stepdown × trigger conditional waterfall | per-rule `condition_trigger` | ⚠️ works but verbose; `WaterfallBranch` is cleaner |
| Floater coupon | `BondDef.coupon_type=FLOATING` + index | ✅ |

The summary: **CRT is structurally the simplest deal in this research corpus, but the existing IR cannot model it correctly because of the missing bond-level loss treatment.** Fixing that one gap unlocks the entire CRT product family (CAS, STACR, CIRT, ACIS).

---

# RMBS — Non-Agency (Subprime, JPMMT 2006)

### Sample read: JPMorgan Mortgage Acceptance 2006 (CWHEQ-style)

This deal is structurally **very different** from agency:

- Two collateral groups (Group 1 / Group 2 mortgage loans).
- **Two parallel waterfalls per period** — Interest Remittance Amount cascade and Principal Remittance Amount cascade — that intersect via OC mechanics.
- 12-step **interest waterfall** plus 5-condition **principal waterfall** (varies by stepdown date and trigger event).
- M-1 through M-10 subordinate stack.
- Excess interest cascade ("Net Monthly Excess Cashflow") with its own multi-step waterfall.
- Net WAC reserve fund with its own sub-waterfall.

#### Interest waterfall (12 numbered steps)

1. Senior interest (concurrent / pro-rata across A-1A, A-1B, A-2..A-5)

2. Senior unpaid interest shortfalls (concurrent) 3-12. M-1 through M-10 interest, **one step per class**, in strict seniority order.

The prospectus phrases steps 3-12 as one step per mezz class — **NOT** as one rule with [M-1..M-10]. The reason: each step is its own waterfall priority where remaining funds at that step depend on what was paid above. (For interest where shortfalls are tracked per-class, step-by-step is more readable.) **Either rendering (per-class steps OR a single multi-target sequential rule) produces identical math** — but the prospectus authors prefer per-class for clarity in the mezz stack.

**Principal waterfall — gated by 2 conditions, 6 conditional blocks**

Each block applies under a different **(stepdown date, trigger event)** combination:

- **A:** prior to stepdown OR Trigger Event in effect → Group 1 Principal → Group 1 Certs first, then Group 2 Certs.
- **B:** prior to stepdown OR Trigger Event in effect → Group 2 Principal → Group 2 Certs first, then Group 1 Certs.
- **C:** prior to stepdown OR Trigger Event in effect → remaining combined principal → M-1, M-2, …, M-10 sequentially.
- **D:** post-stepdown AND no Trigger Event → Group 1 Senior Principal Distribution Amount (a different formula) → Group 1 Senior + cross-allocation to Group 2.
- **E:** post-stepdown AND no Trigger Event → Group 2 Senior Principal Distribution Amount → Group 2 Senior + cross-allocation.
- **F:** post-stepdown AND no Trigger Event → remaining → mezz with **Combined Class M-1, M-2 and M-3 Principal Distribution Amount** (a SHARED computed amount that pays all three sequentially).

**Computed distribution amounts**

Heavy use of NAMED FORMULAS:

- "Group 1 Principal Distribution Amount" = pool collections + advances
- "Group 1 Senior Principal Distribution Amount" = capped formula
- "Combined Class M-1, M-2 and M-3 Principal Distribution Amount" = formula

These are not rules — they are **named scalar calculations** computed each period and referenced by rules. The current IR has `CalculationNode` for trigger metrics but doesn't use it for distribution amounts.

**Sequential Trigger Event vs Trigger Event**

There are **multiple distinct triggers** controlling different behaviors:

- "Trigger Event" — gates the stepdown.
- "Sequential Trigger Event" — gates Class A-1A / A-1B concurrent-vs-sequential.

Both are computed each period from cumulative loss + delinquency ratios with **time-dependent thresholds** (a different threshold table per Distribution Date range).

#### Loss allocation — separate cascade

Loss allocation is a SEPARATE waterfall: realized losses go first to M-10 → M-9 → … → M-1 → seniors. **Reverse seniority order**, not the cash distribution order. Currently the IR has no first-class loss-allocation primitive; it's bolted onto bond writedown logic.

#### Excess interest cascade

"Net Monthly Excess Cashflow" — what's left after paying interest + fees + scheduled principal — has its OWN multi-step waterfall:

1. To OC build until target reached
2. To unpaid interest shortfalls
3. To realized losses (write-up M-10..M-1)
4. To net WAC carryover
5. … and several more

This is a sub-waterfall on a derived stream. The IR's `SPLIT_CASH` primitive can route to a "Net Monthly Excess Cashflow" stream, and then a sequence of rules consumes it. Workable, but requires expressing the cascade explicitly.

---

# Auto ABS — Prime (Ford Credit Auto Owner Trust 2024-C)

## Pre-acceleration priority of payments (single 11-step cascade)

```
1. Transaction Fees and Expenses (capped $375K/yr) — to indenture
trustee, owner trustee, ARR
2. Servicing Fee — to servicer
3. Class A Note Interest — pro rata across Class A notes
4. First Priority Principal Payment — to Class A, sequentially by
class,
   amount = MAX(0, Class A principal - adjusted pool balance)
5. Class B Note Interest
6. Second Priority Principal Payment — to Class A+B, sequentially
by class,
   amount = MAX(0, (A+B principal) - adjusted pool balance) -
step 4 amount
7. Class C Note Interest
8. Reserve Account Replenishment — to top up to original balance
9. Regular Principal Payment — sequentially by class,
   amount = GREATER OF (a) Class A-1 principal,
                       (b) notes principal - (pool balance -
target OC)
   reduced by first + second priority amounts
10. Additional Fees and Expenses (uncapped overflow from step 1)
11. Residual Interest — to depositor
```

## Key structural features

1. **Single Available Funds source** — no group split. All collections pool together; the waterfall consumes them in 11 steps.

2. **Interest and principal interleaved** by class seniority, not as separate sub-waterfalls. Step 3 = A interest. Step 5 = B interest. Step 7 = C interest. The 4 / 6 / 9 between are principal acceleration / catch-up steps.
3. **Computed principal amounts**:
    - "First Priority Principal Payment" — explicit formula based on overcollateralization deficit; only nonzero if Class A principal exceeds adjusted pool balance.
    - "Second Priority Principal Payment" — same shape, A+B view.
    - "Regular Principal Payment" — explicit MAX-of-two-formulas with reductions for upstream payments.
4. **Reserve account replenishment as a numbered step** (step 8). The current IR has `PAY_TO_RESERVE` rule type, so this is covered, but the name is hidden from the user in the FNR fixture (no reserve in agency).
5. **Sequential by class for principal**, but **pro rata within class** for interest. Class A has sub-classes A-1, A-2a, A-2b, A-3, A-4; principal pays A-1 first to retired, then A-2a, etc.; interest pays all 5 pro rata each period.
6. **Capped fees with overflow**: Step 1 caps at $375K/year; any excess fees move to step 10 (after principal). The IR's `max_amount_fixed` covers the cap; the carry-to-later-step needs thought.
7. **Post-acceleration priority of payments**: a SEPARATE waterfall for after Event of Default (not detailed in our extract). Adds an ALTERNATE TOP-LEVEL waterfall gated by deal state.

### What's notable vs RMBS

- **No PAC / TAC / Z**. Auto deals are pure sequential.
- **Single pool, no groups**.
- **No stepdown gate** the way RMBS has it; instead the auto structure relies on the priority principal mechanism (steps 4 + 6) to maintain seniority backstop, and the "Regular Principal" formula (step 9) to build target OC over time.
- **Reserve account is a first-class participant** in the waterfall, not just a credit enhancement footnote.
- **Asset-rep fees** (a relatively recent addition tied to ABS Rule 15Ga-1) appear as a discrete payee.

---

## Auto ABS — Subprime (Santander Drive Auto Receivables Trust 2024-2)

### Pre-acceleration priority of payments (single 14-step cascade)

```
1.  first       — trustee + Delaware trustee + owner trustee +
ARR fees
                  ($300K/yr cap)
2.  second      — servicing fee
3.  third       — Class A interest, pro rata
4.  fourth      — First Allocation of Principal
5.  fifth       — Class B interest
6.  sixth       — Second Allocation of Principal
7.  seventh     — Class C interest
8.  eighth      — Third Allocation of Principal
9.  ninth       — Class D interest
10. tenth       — Fourth Allocation of Principal
```

```
11. eleventh    — reserve replenishment (to Specified Reserve
Balance)
12. twelfth     — Regular Allocation of Principal
13. thirteenth  — trustee fee overflow (per-annum cap exceeded)
14. fourteenth  — residual to certificateholders, pro rata
```

### Same shape as prime auto with parametric differences only

The waterfall is structurally **identical** to Ford Credit prime auto. The differences are entirely parametric:

| Feature | Ford Credit (prime) | SDART (subprime) |
|---|---|---|
| Step count | 11 | 14 |
| Mezz classes | A, B, C | A, B, C, D |
| Named principal allocations | First, Second, Regular | First, Second, Third, Fourth, Regular |
| Trustee fee per-annum cap | $375K | $300K |
| Residual recipient | "Holder of residual interest" | "Certificateholders, pro rata" |

**Implication for the IR:** prime + subprime auto share **one waterfall grammar**. The IR does not need a "subprime auto" rule type or a "prime auto" rule type — same primitives, more parameters.

### Post-acceleration priority of payments

After Event of Default + acceleration, a SEPARATE waterfall applies (typically: trustee fees → all classes interest pro rata → all classes principal sequentially with no priority principal acceleration). Triggered by indenture conditions, not period-by-period.

This is structurally the same as the JPMMT "Trigger Event in effect" branch in the RMBS principal waterfall — an alternate mode gated by a deal-state trigger.

---

# Cross-asset-class observations

### What ALL deals have in common

1. **Numbered priority of payments** — the prospectus is always a numbered list of steps.
2. **Each step has multiple targets in a list with explicit order semantics** (sequential, pro-rata, concurrent).
3. **Each step has explicit cap semantics**: "to its planned balance" / "until retired" / "to zero" / "amount equal to…"
4. **Triggers gate behavior**, but the *type* of trigger varies: stepdown date + cumulative loss + delinquency in RMBS; acceleration / event of default in auto.
5. **A "named amount" abstraction** is always implied — every deal defines named formulas like "Group X Principal Distribution Amount", "First Priority Principal Payment", "Net Monthly Excess Cashflow", and refers to them by name in subsequent steps.
6. **Residual / equity / depositor** is always the final step.

## What VARIES across asset classes

| Feature | Agency MBS | Non-Agency RMBS | Prime Auto |
|---|---|---|---|
| Distinct collateral groups | YES (1-9 per deal) | YES (typically 1-2) | NO |
| Separate INT/PRIN sub-streams | YES (INT_CASH/ PRIN_CASH) | YES (Interest Remittance Amount + Principal Remittance Amount) | NO (combined Available Funds) |
| Group-aware allocation | YES | YES (Group 1 Certs, Group 2 Certs) | N/A |
| PAC / TAC / Z behavior | YES | NO | NO |
| Stepdown date gate | NO | YES | NO |
| OC / Reserve account in waterfall | RARE (REMIC trust-level fees) | YES (excess interest cascade) | YES (step 8) |
| Sequential vs pro-rata switch | NO | YES (Sequential Trigger Event) | NO |
| Interleaved I/P by class | NO | NO (separate I and P waterfalls) | YES (interest 3, prin 4; interest 5, prin 6…) |
| Loss allocation cascade | rare | YES (reverse seniority) | rare (covered by OC) |
| Computed (named) distribution amounts | LIGHT (just “Group N Cash Flow Distribution Amount”) | HEAVY (every step references named formulas) | MEDIUM (priority principal payments are formulas) |
| Recursive splits | YES (FNR 2019-17 Group 7) | RARE | NO |
| Aggregate Group bond bundles | YES | NO | NO |

## Cross-cutting needs

1. **Multi-target rules with explicit payment_style** — already supported in IR. **Use it more** in the FNR fixture and the irGenerator.
2. **Named distribution amounts** — referenced by name in rule sources. Currently the IR has `INT_CASH` / `PRIN_CASH` as built-in streams and `SPLIT_CASH` to create new ones; this generalizes to “any named formula”. Need a `CalculationNode`-style “ComputedAmount” object that produces a per-period scalar usable as a `from_source` cap.
3. `if / elif / else` **over rule blocks** — the RMBS principal waterfall has six mutually exclusive blocks (A through F) keyed on (stepdown date, trigger event). Currently expressed via `condition_trigger` on each rule, which works but is verbose and error-prone (every rule must remember to invert the condition for the “else” branch). A `WaterfallBranch` node with ordered `if / elif / else` cases reads exactly like the prospectus.
4. **Bond-level loss treatment** — the existing `PAY_WRITEDOWN` rule already does the reverse-seniority cascade. What’s missing is the bond’s *response* to a writedown: does the bond’s balance decrease (`WRITEDOWN`) or stay constant with a deferred carryover (`NOTIONAL_HOLD`)? Critical for CRT (writedown is the primary economic event), important for non-agency RMBS subs.

5. **Aggregate Group abstraction** — a NAMED collection of bonds treated as a unit (own planned balance, internal allocation rule). Currently expressed by tagging each bond and listing them in a rule's `to_targets`; works but loses the "this is a group" intent.
6. **Reserve account integration** — already supported via `PAY_TO_RESERVE` / `PAY_FROM_RESERVE_*`. Used heavily in auto; present in non-agency RMBS via OC mechanics.
7. **Excess interest sub-cascade** — a separate sub-waterfall driven by what's left after the "main" interest cascade. Need a clean way to express the carry-forward.

---

# Gaps in the current IR

After this 4-deal sample:

| Gap | Severity | Affects |
|---|---|---|
| Named distribution amounts (computed scalars) referenced by source name | **HIGH** | non-agency RMBS, prime auto |
| Multi-target sequential rules under-used in fixtures and irGenerator | **HIGH** | every asset class (visual / authoring) |
| Aggregate Group bond bundles | MEDIUM | agency MBS |
| Recursive / nested splits | MEDIUM | agency MBS (FNR 2019-17 pattern) |
| Bond-level loss treatment (writedown vs notional-hold) | **HIGH** | CRT, non-agency RMBS, future CMBS |
| Conditional waterfall blocks (`if / elif / else` over rule groups) | MEDIUM | non-agency RMBS, CRT, auto post-acceleration |
| Net WAC reserve / sub-cascade plumbing | LOW (covered by SPLIT_CASH + accounts) | non-agency RMBS |
| Post-acceleration alternate waterfall | LOW (covered by triggers) | auto |
| Capped fee with overflow to later step | LOW | auto |

---

# Proposed IR additions (DRAFT — not approved)

### A. Multi-target rule consolidation (no schema change, fixture rewrite + synth/irGen change)

Stop fragmenting by emitting one rule per bond. Both the FNR fixture authors and the `emitPacTacSchedule` block generator should emit multi-target rules (`to_targets: [PA, PB, PC, PD, EO]`, `payment_style: SEQUENTIAL`, `cap_mode: PLANNED`).

**Why first:** zero schema risk, biggest immediate benefit. The rule count drops from ~24 to ~5 for FNR Group 1.

### B. ComputedAmount node (small schema add)

```python
class ComputedAmountNode(BaseModel):
    name: str            # "Group 1 Principal Distribution Amount"
    expression: str      # "principal + advance_prin + recovery"
    description: str
```

Rules can reference `ComputedAmountNode` names in `from_sources` the same way they reference `INT_CASH`/`PRIN_CASH`. The runtime resolves the name to the per-period scalar.

### C. WaterfallBranch — `if / elif / else` over rule blocks

The natural way to express "if Trigger Event in effect: pay rules 1; else: pay rules 2" is exactly that — an explicit `if / elif / else` chain at the IR level, not per-rule `condition_trigger` tags. Per-rule tagging works mathematically but is error-prone (you have to remember to invert the condition on every "else" rule and keep the inversion in sync) and unreadable (the prospectus phrasing "If X, do Y, else do Z" is fragmented across many rules).

```python
class WaterfallBranch(BaseModel):
    branch_id: str
    description: str
    cases: list[WaterfallCase]   # ordered; first matching case fires

class WaterfallCase(BaseModel):
    when: str | None             # trigger name, None for the `else` arm
    invert: bool = False         # treat the trigger as `not when`
    expr: str | None
        # OR a free expression evaluated per period
    rules: list[RuleNode]        # rules to execute when this case matches
```

Reads exactly like the prospectus:

```yaml
waterfall_rules:
  - branch_id: principal_waterfall
    description: "RMBS principal allocation: stepdown × trigger event"
    cases:
      - when: TriggerEvent
        rules: [ ... rules block A: sequential to senior, no mezz ... ]
      - when: StepdownDate
        invert: true
        rules: [ ... block B: pre-stepdown sequential ... ]
      - expr: "stepdown_date_reached and not trigger_event"
        rules: [ ... block C: post-stepdown pro-rata ... ]
```

```
    - when: null   # the `else` arm
      rules: [ ... default block ... ]
```

**Why prefer this over per-rule** `condition_trigger`**:**

1. **Mutual exclusion is explicit.** `if/elif/else` semantics guarantee exactly one branch fires. Per-rule tags can accidentally fire two branches if conditions aren't perfectly complementary.
2. **DRY.** Don't repeat the same condition on 14 rules.
3. **Reads like the prospectus.** "Block A — Trigger Event in effect" maps to one case; "Block B — pre-stepdown" to another.
4. **Refactor-friendly.** Adding a rule to a conditional block is one append; under the per-rule scheme it's an append plus matching the condition exactly.

**When to still use per-rule** `condition_trigger`**:** one-off gates on a *single* rule (e.g., "this single fee only applies if the servicer is in default"). Per-rule remains for one-off cases; `WaterfallBranch` is for multi-rule blocks.

This is the IR equivalent of asking "why don't we just write `if/else`?" — the answer is "we should, and that's what this node is."

### D. AggregateGroup bond bundle

```
class AggregateGroupDef(BaseModel):
    name: str            # "Aggregate Group III"
    members: list[str]   # ["MA", "ME", "MC", "MG", "MB"]
    schedule_contract: list[...] | None    # planned balance
        (PAC)
    internal_allocation: PaymentStyle      # SEQUENTIAL /
        PRO_RATA
```

A virtual bundle that rules can target by name. The runtime caps at the bundle's planned balance and distributes internally per the allocation rule.

### E. Bond-level loss treatment (the real loss-allocation question)

Reverse-seniority loss allocation isn't a special new IR construct — it's the standard pattern across RMBS, CRT, and future CMBS, and the existing `PAY_WRITEDOWN` rule type already expresses it:

```
rule_type: PAY_WRITEDOWN
from_sources: [LOSS]
to_targets: [B-2, B-1, M-2, M-1]   # reverse seniority
payment_style: SEQUENTIAL
```

The **real question** that the IR currently doesn't answer is: **when a loss hits a bond, what happens to that bond's balance and accrual base going forward?** Two distinct treatments exist in real prospectuses:

1. `WRITEDOWN` — bond balance is reduced by the loss amount. Future coupon accrues on the *reduced* balance. Future principal distributions are based on the reduced balance. This is the CRT default and the non-agency RMBS subordinate default.

2. NOTIONAL_HOLD — bond balance stays unchanged. The loss becomes a deferred-amount carryover that reduces cash to that bond until covered by recovery / excess interest, but the bond continues to accrue coupon on its *full* original balance. This is rare but appears in some prime jumbo deals and in particularly investor-friendly subordinate structures.

3. NONE — bond doesn't absorb losses at all (typically senior-most class with full guarantee).

Adding this is a small BondDef extension:

```
class LossTreatment(StrEnum):
    WRITEDOWN = "WRITEDOWN"
    NOTIONAL_HOLD = "NOTIONAL_HOLD"
    NONE = "NONE"

class BondDef(BaseModel):
    ...
    loss_treatment: LossTreatment = LossTreatment.NONE
    writeup_enabled: bool = False   # CRT-style loss reversal
```

The runtime change is small: in PAY_WRITEDOWN, look up each target bond's loss_treatment and either decrement its balance (WRITEDOWN) or accumulate a deferred amount (NOTIONAL_HOLD). Recovery / writeup applies inversely.

**Why this matters for CRT.** CRT bonds are pure notional-bearing instruments — there is no actual cash collateral in the trust. The bonds' economic existence IS their balance, and loss allocation IS the primary cashflow event. Without loss_treatment on BondDef, the IR can't model CRT at all. The existing PAY_WRITEDOWN rule type is the *cascade*; the missing piece is the *bond's* response to that cascade.

### F. Recursive SPLIT_CASH (no schema change, runtime change)

Already supported in principle: a SPLIT_CASH target stream can be the source of another SPLIT_CASH. Verify this works for FNR 2019-17 Group 7's nested 16.67 / 83.33 / first / second pattern.

---

## Open questions for user before any code change

1. **Priority.** Which of (A)-(F) above matter for your near-term deal universe? My read: (A) is must-have *now* (it fixes the visual problem you flagged); (B) and (C) are needed once we start modeling private-label RMBS; (D) is agency-MBS-specific;

    1. is **must-have for CRT** and important for non-agency RMBS subordinates; (F) is already kind of working.

2. **More research breadth.** Do you want me to extend this study to the other asset classes (subprime auto, credit card, CLO, marketplace, equipment, solar) before proposing changes, or are the structural patterns above enough to converge on the abstractions for now and revisit the IR when those asset classes are needed? **My recommendation: do (A) now (no IR risk), park (B)-(F) until we have ≥5 deals per relevant asset class.**

3. **The PAC/TAC/Z question** you raised originally. The IR *intentionally* does not have PAY_PRINCIPAL_PAC_SCHEDULE as a distinct rule type — the PAC behavior is

encoded as `BondDef.tranche_behavior=PAC` + `BondDef.schedule_contract` + `RuleNode.cap_mode=PLANNED`. This is the right call for the *runtime* (PAC and SEQ have the same allocation logic; only the bond's per-period cap differs). But for the *UI* we should recognize "this rule pays a PAC bond with cap_mode=PLANNED" and render it via the existing `pay_pac_schedule` Blockly block. The synthesizer can detect this from the IR (any rule whose targets include a bond with `tranche_behavior=PAC` and whose `cap_mode=PLANNED` is a PAC schedule rule).

**The IR doesn't need a new rule type for PAC/TAC; the synthesizer + irGenerator just need to recognize the pattern.**

---

# Additional deals (round 2 research)

## Ginnie Mae REMIC 2025-203 (single-family, multi-group)

Three Security Groups; Group 3 has the canonical PAC + Z + Support structure verbatim:

> **SECURITY GROUP 3** — The Group 3 Principal Distribution Amount and the Accrual Amount will be allocated in the following order of priority: 1. Sequentially, to BP and PB, **in that order, until reduced to their Aggregate Scheduled Principal Balance for that Distribution Date** 2. To Z, until retired 3. Sequentially, to BP and PB, **in that order, without regard to their Aggregate Scheduled Principal Balance, until retired**

Identical structure to FNR. "Aggregate Scheduled Principal Balance" is the same abstraction as Fannie's "Aggregate Group Planned Balance". Confirms the FNR pattern is industry-standard for agency REMICs.

## Ginnie Mae REMIC 2025-009 (HECM-backed reverse mortgage)

Reverse-mortgage REMIC has a distinct structural feature — the **Deferred Interest Amount**. The 3-step waterfall is:

> 1. Concurrently, to AI, FA and FB, **pro rata based on their respective Interest Accrual Amounts**, up to the Class Interest Accrual Amount
> 2. Concurrently, to FA and FB, **pro rata based on their Class Principal Balances**, in reduction of their Class Principal Balances, up to the Principal Distribution Amount
> 3. To AI, until the **Class AI Deferred Interest Amount** is reduced to zero

Step 3 is unique — it pays accrued-but-unpaid IO interest only *after* the principal cascade. Standard MBS pays IO at the same time as bond interest. **Reverse-mortgage REMICs need a "deferred interest catch-up" rule type or a `cap_mode=DEFERRED` flag.** Not in current IR.

## Ginnie Mae REMIC 2024-115 (Multifamily)

> Allocation of Principal: On each Distribution Date, a percentage of the Principal Distribution Amount will be applied to the **Trustee Fee**, and the remainder of the Principal Distribution Amount (the **"Adjusted Principal Distribution Amount"**) will be allocated, sequentially, to A and B, in that order, until retired.

Multifamily REMICs **deduct trustee fees from principal** as a percentage *before* the cascade. Single-family agency REMICs don't do this (trustee/servicer fees are netted at the underlying MBS layer). The IR's `PAY_FEE` rule with `basis_type=COLLATERAL_BALANCE` covers this; the multifamily case puts the fee earlier in the cascade.

## Freddie Mac REMIC general structure (offering circular)

Freddie Mac REMICs introduce a structural concept that doesn't exist in our current IR: **Single-Tier vs Double-Tier Series**:

- **Single-Tier**: REMIC Certificates represent direct beneficial ownership in *one* REMIC Pool. Cashflows flow pool → classes.
- **Double-Tier**: An **Upper-Tier REMIC Pool** + one or more **Lower-Tier REMIC Pools** stacked. Lower-Tier Classes are themselves **Mortgage Securities** of the Upper-Tier Pool. Cashflows flow pool → Lower-Tier classes → Upper-Tier classes.

This is **REMIC-inside-a-REMIC** structure. Most agency deals abstract this away (the cashflow analyst just looks at the ultimate classes). For our IR, we already model this implicitly — the `Loan → BMA cashflow engine → DealRunInput` chain treats the underlying agency MBS as a black box delivering a single pool's cashflows to the deal. The lower-tier internal mechanics are out of scope unless we want to model RCR / MACR exchanges.

Also notable: **MACR (Modifiable and Combinable REMIC) Certificates** allow exchange of one class for proportionate interests in another. This is purely an issuance/secondary mechanic (no waterfall implication) and doesn't touch the IR.

## Verus Securitization Trust 2024-9 (Modern Non-QM RMBS)

Adds three patterns not in JPMMT 2006:

1. **Step-up coupon at year 5.** "On each payment date beginning in January 2029, the A1, A2 and A3 [classes] will receive the sum of [the deal's] fixed coupon, plus 1.00%."
   - Time-conditional coupon change. Requires the IR to support coupon as a function of period, not a static field.
   - The current `BondDef.coupon: float` does not capture this. `BondDef.coupon_schedule: list[{period, rate}]` would.
2. **Last Cashflow (LCF) class.** "A1A, A1B, A1LCF" — A1LCF gets paid LAST among the seniors. New seniority sub-pattern.
   - LCF works as a normal bond with a junior position within the senior tier; expressible via the existing rule ordering.
3. **Class XS** — explicit excess-spread tracking class. Class XS noteholders also control optional-redemption rights.
   - Excess-spread tracking is just a residual class with extra economic rights; expressible via `tranche_type=RESIDUAL`.
   - Optional-redemption is governance, not waterfall, and is out of scope for the IR.

Pay structure: "senior notes pro rata, mezz + sub sequentially." Same hybrid pattern as JPMMT 2006 post-stepdown, but encoded with no stepdown date — Non-QM deals typically don't have one because the loans amortize quickly.

## Toyota Auto Receivables 2024-A (Prime Auto, Toyota)

Confirms the Ford Credit prime auto shape. Differences:

- Has **First Priority Principal Distribution Amount** (Class A catch-up) AND **Second Priority Principal Distribution Amount** (Class A+B catch-up) AND **Regular Principal Distribution Amount** (target OC build) — same three named amounts.
- "**Yield Supplement Overcollateralization Amount**" (YSOC) — a parallel concept used in adjusting the "adjusted pool balance" for sub-WAC loans. Specific to auto deals where some loans have rates below the WAC needed to support the bonds. Pure computation tweak, doesn't change waterfall structure.
- Reserve account replenishment, capped trustee fees with overflow — same as Ford / SDART.
- Confirms prime auto = same grammar across sponsors.

## Toyota Lexus Owner Trust 2024-A (Auto LEASE — different shape)

Lease-backed ABS has a SIMPLER waterfall (8 steps, not 11-14):

```
1. Servicing Fee
2. Transaction Fees and Expenses (capped $300K/yr)
3. Note Interest (pro rata across all classes — NOT class-by-class)
4. Note Principal — priority principal distribution amount
   (single combined amount, not per-class)
5. Reserve Account Deposit
6. Note Principal — regular principal distribution amount
7. Additional Transaction Fees and Expenses (overflow)
8. Excess Amounts to certificateholder
```

**Differences vs auto loan ABS:**

- **Pro-rata interest across ALL classes** (no class-by-class step). Less common in loan ABS.
- **One combined "priority principal distribution amount"** instead of per-class first/second priority. The lease structure pays all notes pro rata for principal, with no class-priority cascade.
- **"Securitization Value"** instead of "Pool Balance" — the lease collateral is valued differently because of residual risk on the vehicles.
- Otherwise structurally the same as auto loan ABS.

The IR already supports this (multi-target PAY_INTEREST with PRO_RATA payment_style; multi-target PAY_PRINCIPAL with PRO_RATA). Just simpler.

## Westlake Automobile Receivables Trust 2024-1 (Subprime Auto)

8 classes of notes (A-1, A-2, A-3 + B, C, D, E). Same waterfall shape as SDART — interleaved I/P, named priority allocations, reserve replenishment as a step. Westlake confirms the subprime auto pattern is consistent across sponsors.

## Updated cross-asset matrix

After 13 deals across 5 asset classes:

| Feature | Agency MBS | Agency CRT | Non-Agency RMBS | Prime Auto | Subprime Auto | Auto Lease |
|---|---|---|---|---|---|---|
| Distinct collateral groups | YES (1-9) | NO (single ref pool) | YES (1-2) | NO | NO | NO |
| Separate INT/PRIN sub-streams | YES | YES (synthetic) | YES | NO | NO | NO |
| PAC / TAC / Z behavior | YES | NO | NO | NO | NO | NO |
| Stepdown date gate | NO | YES | YES | NO | NO | NO |
| Reserve account in waterfall | RARE | NO | YES | YES | YES | YES |
| Sequential vs pro-rata switch | NO | YES (gate on perf trigger) | YES | NO | NO | NO |
| Interleaved I/P by class | NO | NO | NO | YES | YES | NO (pro-rata) |
| Bond writedown on losses | NO | **YES (core mechanic)** | YES | NO | NO | NO |
| Bond writeup on loss reversal | NO | YES (rare) | RARE | NO | NO | NO |
| Named computed distribution amounts | LIGHT | MEDIUM | HEAVY | MEDIUM | MEDIUM | LIGHT |
| Recursive splits | YES | NO | RARE | NO | NO | NO |
| Aggregate Group bond bundles | YES | NO | NO | NO | NO | NO |
| Step-up coupon | RARE | NO | YES (Non-QM) | NO | NO | NO |
| Deferred interest catch-up | RARE (HECM) | NO | NO | NO | NO | NO |
| Acceleration alternate waterfall | NO | NO | NO | YES | YES | YES |

## Updated IR gap list

Adding the new patterns from round 2:

| Gap | Severity | Affects | Already in IR? |
|---|---|---|---|
| | **HIGH** | every class | |

| Gap | Severity | Affects | Already in IR? |
|---|---|---|---|
| Multi-target rule consolidation (authoring) | | | NO (fixture/irGen issue, not IR) |
| Named computed distribution amounts | **HIGH** | non-agency RMBS, auto | NO |
| Aggregate Group bond bundles | MEDIUM | agency MBS | NO |
| Recursive SPLIT_CASH | MEDIUM | agency MBS | PARTIAL (need runtime verify) |
| **Bond-level loss treatment** (`writedown` vs `notional_hold`) | **HIGH** | CRT (core), non-agency RMBS | NO |
| Conditional waterfall blocks (`if / elif / else`) | MEDIUM | non-agency RMBS, CRT, auto post-accel | PARTIAL (per-rule trigger only) |
| Bond writeup on loss reversal | LOW | CRT | NO |
| **Time-conditional bond coupons (step-ups)** | **MEDIUM** | Non-QM, callable seniors | NO |
| **Acceleration alternate waterfall** | **MEDIUM** | every auto deal | PARTIAL (trigger-based) |
| **Deferred interest catch-up** (HECM IO) | LOW | reverse mortgage REMIC | NO |
| Net WAC reserve sub-cascade | LOW | non-agency RMBS | PARTIAL (SPLIT_CASH + accounts) |
| Capped fee with overflow to later step | LOW | auto | PARTIAL (max_amount_fixed) |

# Part 2 — IR reference: the schema and how to write a deal

This second half of the document covers the IR itself. Goal: make the IR both **highly reusable** (one schema for many asset classes) and **human-readable** (an analyst should be able to read the IR top-to-bottom and match it to the prospectus paragraph by paragraph).

## IR overview

The IR is a JSON-serializable Pydantic schema that captures one deal's entire static structure in a single document. It is:

- **Source of truth.** The runtime executes the IR; the UI (Blockly canvas + Properties panel) edits the IR; saved deals persist as IR JSON; tests compare against the IR.
- **Versioned.** `schema_version: "1.0.0"` allows forward-compatible migrations.

- **Self-validating.** Pydantic enforces field types; the top-level `_validate_references` validator enforces cross-field consistency (every rule's `from_sources` and `to_targets` must reference declared bonds, accounts, fees, or built-in streams; every rule's `condition_trigger` must reference a declared trigger; every bond's `support_tranches` must reference real bonds; etc.).
- **Round-tripping.** IR → runtime → cashflow output is deterministic; IR → Blockly synthesizer → workspace → irGenerator → IR is meant to be lossless.

## Top-level schema: `DealDefinition`

```
class DealDefinition(BaseModel):
    schema_version: str = "1.0.0"
    deal_name: str                          # "FNR 2006-018 (Group
        1 + Group 2)"
    description: str = ""
    origination_date: date | None
    settlement_date: date | None

    # Structure
    bonds: list[BondDef]                    # All tranches,
        including pseudo bonds
    accounts: list[AccountDef]
        # Reserves, prefunding, custodial
    fees: list[FeeDef]                      # Trustee, servicer,
        etc.
    triggers: list[TriggerNode]             # Conditional gates
    calculations: list[CalculationNode]
        # Named expressions for triggers
    waterfall_rules: list[RuleNode]         # The actual priority
        of payments
    collateral_groups: list[CollateralGroupDef]  # Multi-pool
        deals

    # Solver / overrides / runtime extensions (see below)
    deal_knobs: dict[str, Any] = {}
```

### `deal_knobs` — what's actually in there

`deal_knobs` is intentionally a free-form `dict[str, Any]` because it serves four distinct purposes. **Five reserved keys** have specific runtime meaning; everything else is a free scalar that gets injected into expression-evaluation context.

| Key | Type | Purpose |
|---|---|---|
| `source_formulas` | `dict[str, str]` | Named per-period expressions that become first-class `from_source` tokens. Example: `{"NET_EXCESS": "INT_CASH - bond_coupon_total"}` makes `NET_EXCESS` |

| Key | Type | Purpose |
|---|---|---|
| | | referenceable in any rule's `from_sources`. |
| `balance_trackers` | `dict[str, str]` | Maps a residual / pseudo bond's name to a runtime quantity it should mirror. Example: `{"R": "collateral_balance"}` makes residual `R` carry a balance equal to outstanding pool balance each period. Used when a residual interest's economics track a non-bond quantity. |
| `orig_collat_bal_override` | `float` | Override for the original collateral balance used in trigger denominator calculations (when the natural pool balance differs from the trigger reference balance, e.g., tape excludes some loans). |
| `allow_negative_cashflow_math` | `bool` | Permit transient negative intermediate cash states during rule execution. Used for deals with negative-amortization streams (HECM, certain ARM cases) where cash conservation invariants need temporary relaxation. |
| <any_identifier> | `int \| float` | **Free expression-context globals.** Every numeric value with an identifier-safe key gets injected into the `expr` evaluation context for fee `amount_expr`, rule `max_amount_expr`, calculation `expression`, and trigger thresholds. Example: `{"servicing_fee_bps": 25.0}` lets a fee expression reference `servicing_fee_bps / 10000` directly. |

The four functional roles, in plain language:

1. **Solver scratch-pad.** The solver writes proposed values into deal_knobs.<name> and re-runs the deal. `KnobSpec.knob_path` uses dot-paths like `deal_knobs.class_a_pctbal` to reach them.
2. **Expression-context globals.** Numeric values are auto-injected into the runtime expression evaluator so any expression-bearing field (`amount_expr`, `max_amount_expr`, `condition_expr`, trigger thresholds, calculation expressions) can reference them by bare name without declaring them as `CalculationNode`s.

3. **Runtime feature flags.** A few well-known boolean keys toggle runtime behavior (currently just `allow_negative_cashflow_math`).
4. **Runtime extension hooks.** `source_formulas` and `balance_trackers` are reserved-key escape hatches for capabilities that haven't been promoted to first-class IR fields yet. Both are candidates for elevation: `source_formulas` should become the proposed `ComputedAmountNode`; `balance_trackers` should become a `BondDef` field (`tracks_quantity: str | None`).

### Migration intent for `deal_knobs`

Per the human-readability principle "no hidden runtime knobs," `deal_knobs` is a tension we accept temporarily. The direction of travel is to **shrink** the schemaless surface:

- `source_formulas` → graduate to `calculations: list[ComputedAmountNode]` (proposed addition B).
- `balance_trackers` → graduate to `BondDef.tracks_quantity` field.
- `orig_collat_bal_override` → graduate to a top-level `DealDefinition.original_collateral_balance: float | None`.
- `allow_negative_cashflow_math` → graduate to a `DealDefinition.runtime_options: RuntimeOptions` typed nested model (one field per flag).

After graduation, `deal_knobs` becomes purely #1 + #2: a typed scratch-pad of named scalar overrides for the solver and for expression evaluation, with everything behavioral promoted to real schema fields.

## `BondDef` — every tranche the deal pays

A bond is anything that receives cash from the waterfall. This includes residual classes and "pseudo bonds" used as accounting sinks for fees.

```
class BondDef(BaseModel):
    name: str                                # "PA", "Class A-1",
        "M-1"
    tranche_type: TrancheType
        # PAC | TAC | SUPPORT | SEQUENTIAL | PO | IO | Z_BOND |
        RESIDUAL | PSEUDO
    tranche_behavior: TrancheBehavior        # PAC | TAC | Z |
        SEQUENTIAL | ACCRETION_DIRECTED
    is_bond: bool                            # False for residual/
        pseudo
    is_pseudo: bool                          # True for fee sinks

    # Coupon
    coupon_type: CouponType                  # FIXED | FLOATING |
        ZERO
    coupon: float | None
        # Annual percent (5.5 = 5.5%)
    margin: float | None                     # Floater spread over
        index
```

```
    index_name: str | None
        # SOFR | TERM_SOFR_1M | etc.
    cap: float | None                     # Floater rate cap
    floor: float | None                   # Floater rate floor

    # Sizing
    size_dollars: float | None
    size_pct: float | None                # 0..100

    # Maturity / accrual
    maturity_date: date | None
    day_count: DayCount                   # 30/360 | ACT/360 |
        ACT/ACT
    accrual_period: AccrualPeriod         # MONTHLY | QUARTERLY

    # PAC / TAC parameters
    schedule_type: ScheduleType | None    # PLANNED | TARGETED
    schedule_model_type: PrepayModelType  # PSA | CPR | ABS |
        CUSTOM_VECTOR
    schedule_speed_low: float | None      # PAC lower PSA
    schedule_speed_high: float | None     # PAC upper PSA
    schedule_speed_target: float | None   # TAC pricing PSA
    schedule_contract: list[dict]         # [{period,
        target_balance}]
    schedule_tolerance_bps: float | None
    support_tranches: list[str]           # PAC's support stack
    supported_by_tranches: list[str]      # Z's accretion targets

    # Z-bond / accrual
    z_accrual_enabled: bool
    z_release_trigger: str | None
    accrual_start_period: int | None
    accrual_end_period: int | None

    # Notional / IO
    parent_tranche: str | None
    relation_type: StructureRelation | None  # NOTIONAL_IO |
        INVERSE | etc.
    notional_ratio: float | None
    tracks_bonds: dict[str, list[str]] | None  # {"balance":
        ["DO"]}

    # Multi-group
    group_id: str | None                  # "GROUP_1", "GROUP_2"

    # Solver knob flags
    solver_knob_coupon: bool
    solver_knob_size: bool
```

```
    pay_mode: PayMode                          # CASH_PAY | PIK
```

**Reusability principle:** one `BondDef` covers PAC, TAC, Z, IO, PO, sequential, support, residual, pseudo. The differences are encoded in `tranche_type` + `tranche_behavior` + the schedule / accrual / tracking fields. **No new bond class needs a new schema.**

## `RuleNode` — one waterfall step

```
class RuleNode(BaseModel):
    rule_id: str                               # "r_int_PA_pacI"
    rule_type: RuleType                        # PAY_INTEREST |
        PAY_PRINCIPAL | ...
    order: int                                 # 0, 1, 2 ... priority

    from_sources: list[str]                    # ["INT_CASH"] or
        ["GROUP_1_PRIN_CASH"]
    to_targets: list[str]                      # ["PA", "PB", "PC",
        "PD", "EO"]
    payment_style: PaymentStyle
        # SEQUENTIAL | PRO_RATA | CONCURRENT
    cap_mode: CapMode | None
        # PLANNED | SCHEDULED | TARGETED | NONE

    max_amount_fixed: float | None             # Hard $ cap on this
        rule
    max_amount_expr: str | None                # Computed cap
        expression
    target_weights: list[float] | None
        # SPLIT_CASH per-target weights

    condition_trigger: str | None              # Trigger name
    condition_invert: bool                     # Run when trigger is
        FALSE
    condition_expr: str | None                 # Custom condition
        expression

    reserve_account: str | None                # PAY_TO_RESERVE /
        PAY_FROM_RESERVE_*
    allow_negative_source: bool

    group_id: str | None                       # Multi-group routing
    description: str = ""
```

**Reusability principle:** every prospectus step maps to ONE `RuleNode`. The shape of the step (PAC schedule, sequential pay, pro-rata, fee, residual sweep, conditional pay) is conveyed by the *combination* of `rule_type` + `payment_style` + `cap_mode` + `condition_trigger`. **No new rule type for “PAY_PRINCIPAL_PAC_SCHEDULE”** or similar — those are just

`PAY_PRINCIPAL` with `cap_mode=PLANNED` and the targeted bond's `tranche_behavior=PAC`.

### `RuleType` enum — what cash this rule moves

| RuleType | Cash moved | Typical sources | Typical targets |
|---|---|---|---|
| `PAY_INTEREST` | Bond cash interest | INT_CASH or CASH | One or more bonds |
| `PAY_INTEREST_SHORTFALL` | Catch-up of unpaid interest | INT_CASH | Bonds with unpaid coupon |
| `PAY_PRINCIPAL` | Principal | PRIN_CASH or CASH | One or more bonds |
| `PAY_WRITEDOWN` | Loss allocation | LOSS | Bonds (reverse seniority) |
| `PAY_FEE` | Fee | CASH or any source | One fee payee |
| `PAY_TO_RESERVE` | Reserve top-up | CASH or interest | Reserve account |
| `PAY_FROM_RESERVE_INTEREST` | Reserve cover | Reserve account | Bonds (interest shortfall) |
| `PAY_FROM_RESERVE_PRINCIPAL` | Reserve cover | Reserve account | Bonds (principal acceleration) |
| `PAY_FROM_RESERVE` | Reserve sweep | Reserve account | Bonds or other targets |
| `PAY_RECOURSE_INTEREST` | Recourse cover | Sponsor recourse | Bonds (interest shortfall) |
| `PAY_RECOURSE_PRINCIPAL` | Recourse cover | Sponsor recourse | Bonds (principal acceleration) |
| `PAY_RESIDUAL` | Residual sweep | Any leftover stream | Residual classes |
| `SPLIT_CASH` | Stream plumbing | One source stream | N named virtual streams |

### `PaymentStyle` — order semantics within a multi-target rule

| Style | Behavior |
|---|---|
| `SEQUENTIAL` | Pay first target until its cap, then next, etc. ("In that order") |
| `PRO_RATA` | Pay all targets simultaneously by their balance / face / coupon weight |
| `CONCURRENT` | Synonym of PRO_RATA used in some contexts |

### `CapMode` — schedule cap interpretation for PAY_PRINCIPAL

| Mode | Stops paying when | Prospectus phrasing |
|---|---|---|
| `PLANNED` | Bond reaches its `schedule_contract` planned balance | "to its Planned Balance" |
| `SCHEDULED` | | "to its Scheduled Balance" |

| Mode | Stops paying when | Prospectus phrasing |
|---|---|---|
| | Bond reaches its scheduled (next-period) balance | |
| TARGETED | Bond reaches a target | "to its Targeted Balance" |
| NONE | Never (cleanup rule) | "without regard to its Planned Balance" / "until retired" |

## Built-in source/target tokens (reusable across asset classes)

Tokens that any rule's `from_sources` / `to_targets` can reference without explicit declaration:

| Token | Meaning |
|---|---|
| `CASH` | Combined pool cashflow (interest + principal + recovery) |
| `COLLATERAL` | Alias for `CASH`, common in older deals |
| `INT_CASH` | Pool interest stream only (separated from principal) |
| `PRIN_CASH` | Pool principal stream only |
| `LOSS` | Pool loss stream (for writedown rules) |
| `GROUP_<id>_CASH` | Combined cashflow for collateral group `<id>` |
| `GROUP_<id>_INT_CASH` | Interest stream for collateral group `<id>` |
| `GROUP_<id>_PRIN_CASH` | Principal stream for collateral group `<id>` |
| `GROUP_<id>_COLLATERAL` | Alias for `GROUP_<id>_CASH` |
| `GROUP_<id>_LOSS` | Loss stream for collateral group `<id>` |

When a rule declares `group_id`, the bare tokens (`CASH`, `INT_CASH`, `PRIN_CASH`, `COLLATERAL`, `LOSS`) are auto-prefixed with `GROUP_<id>_` at compile time. So a multi-group rule can write `from_sources: ["INT_CASH"]` and have it resolve to the right group automatically.

# Other elements

### `AccountDef` — reserve / prefunding / custodial accounts

```
class AccountDef(BaseModel):
    name: str                             # "Reserve_Account"
    account_type: AccountType             # RESERVE | PREFUNDING
        | CUSTODIAL | DISTRIBUTION
    starting_amount: float                # $ amount at closing
    starting_pct: float | None            # OR % of pool / bond
        stack
    starting_basis: MinimumBasis          # FIXED_DOLLAR |
        NOTE_BALANCE
    minimum_amount: float                 # Floor
    minimum_pct: float | None
    minimum_basis: MinimumBasis
```

### `FeeDef` — periodic fee paid to a payee

```
class FeeDef(BaseModel):
    name: str                                    # "TRUSTEE", "SERVICER"
    basis_type: FeeBasisType                     # FIXED_DOLLAR |
        COLLATERAL_BALANCE | PER_LOAN
    amount: float                                # $ amount per period
        (FIXED_DOLLAR)
    amount_expr: str | None                      # OR computed
        expression
    rate: float | None                           # Annual rate
        (COLLATERAL_BALANCE)
    rate_expr: str | None
    minimum: float                               # Floor on the fee
    frequency: FeeFrequency
        # MONTHLY | QUARTERLY | ANNUAL
    cumulative: bool                             # Track unpaid fees as
        carryover
```

### `TriggerNode` — conditional gate

```
class TriggerNode(BaseModel):
    name: str                                    # "CumLossTrigger"
    metric_type: TriggerMetricType               # CUMULATIVE_LOSS |
        CUMULATIVE_DEFAULT | DELINQUENCY_RATE | OC_TEST | IC_TEST
        | CUSTOM
    threshold_value: float | None
    threshold_schedule: list[float] | None
        # Per-period threshold (e.g., RMBS time-stepped triggers)
    calculation_ref: str | None                  # Pointer to a
        CalculationNode for dynamic thresholds
    comparison_ref: str | None                   # ">" / ">=" / etc.
    cure_periods: int | None                     # How many clean
        periods to clear the trigger
```

### `CalculationNode` — named expressions

```
class CalculationNode(BaseModel):
    name: str                                    # "cum_loss_pct"
    expression: str                              # "sum(loss[0:i+1]) /
        orig_collat_bal"
    description: str
```

Used by triggers (and in proposed `ComputedAmountNode`) to compute per-period scalars. Supports a safe subset of Python: arithmetic, min/max/abs, references to bond/account/pool state.

### `CollateralGroupDef` — multi-pool deals

```
class CollateralGroupDef(BaseModel):
    group_id: str                          # "GROUP_1"
    label: str                             # Human-readable
    description: str
```

Activates per-group cash routing in the runtime. When non-empty, every non-pseudo bond and every cashflow-touching rule must declare `group_id`.

---

## Worked example: FNR 2006-018 Group 1, written in IR

The prospectus says (verbatim, paraphrased for brevity):

> Group 1 cash flow priority of payments: 1. Pay interest on all cash-paying bonds. 2. Pay principal of Aggregate Group I to its Planned Balance (PA → PB → PC → PD → EO sequential). 3. Pay principal of Aggregate Group II to its Planned Balance (TA → TB sequential). 4. Pay principal to Z to zero. 5. Distribute 95.65% of remaining principal to support sequential (WA → WB → WC → WD → WE → WG); 4.35% to PO. 6. Pay principal of Aggregate Group II to zero (without regard to planned balance). 7. Pay principal of Aggregate Group I to zero (without regard to planned balance). 8. Sweep remaining cash to residual.

Below is the **same waterfall expressed compactly in IR** (one rule per prospectus step). Every rule has a stable `rule_id` that matches the prospectus phrasing so an analyst can search the IR and find the corresponding step:

```yaml
deal_name: "FNR 2006-018 Group 1"
collateral_groups:
  - group_id: GROUP_1
    label: "Group 1 (PAC + Z + Support)"

bonds:
  # Aggregate Group I (PAC I + IO/PO)
  - { name: PA,  tranche_type: PAC,    tranche_behavior: PAC,
        group_id: GROUP_1,
      coupon: 5.5, size_dollars: 33710000, schedule_contract:
        [...],
      support_tranches: [WA, WB, WC, WD, WE, WG, PO] }
  - { name: PB,  tranche_type: PAC,    tranche_behavior: PAC,
        group_id: GROUP_1, ... }
  - { name: PC,  tranche_type: PAC,    tranche_behavior: PAC,
        group_id: GROUP_1, ... }
  - { name: PD,  tranche_type: PAC,    tranche_behavior: PAC,
        group_id: GROUP_1, ... }
  - { name: EO,  tranche_type: PO,     coupon_type: ZERO,
        group_id: GROUP_1, ... }
  - { name: EI,  tranche_type: IO,     tracks_bonds: { balance:
        [PA, PB, PC, PD] }, ... }
```

```yaml
  # Aggregate Group II (PAC II / accretion-directed)
  - { name: TA,  tranche_type: PAC,    tranche_behavior: PAC,
      group_id: GROUP_1, ... }
  - { name: TB,  tranche_type: PAC,    tranche_behavior: PAC,
      group_id: GROUP_1, ... }

  # Z-bond (Aggregate Group II support)
  - { name: Z,   tranche_type: Z_BOND, tranche_behavior: Z,
      pay_mode: PIK,
    group_id: GROUP_1, z_accrual_enabled: true,
      supported_by_tranches: [TA, TB] }

  # Support tranches
  - { name: WA,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WB,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WC,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WD,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WE,  tranche_type: SUPPORT, group_id: GROUP_1, ... }
  - { name: WG,  tranche_type: SUPPORT, group_id: GROUP_1, ... }

  # Support PO (4.35% of support cash)
  - { name: PO,  tranche_type: PO,     coupon_type: ZERO,
      group_id: GROUP_1, ... }

  # Residual
  - { name: R,   tranche_type: RESIDUAL, is_pseudo: true }

waterfall_rules:
  # Step 1 — Interest cascade (one rule, all cash-paying bonds in
      priority order)
  - rule_id: r_int_cascade
    rule_type: PAY_INTEREST
    order: 0
    group_id: GROUP_1
    from_sources: [INT_CASH]
    to_targets: [PA, PB, PC, PD, TA, TB, EI, WA, WB, WC, WD, WE,
      WG]
    payment_style: SEQUENTIAL
    description: "Pay accrued bond coupon. Z is PIK; its coupon
      is capitalized."

  # Step 2 — PAC I to its Planned Balance (one rule, all PAC I
      bonds in priority)
  - rule_id: r_prin_pac_i_planned
    rule_type: PAY_PRINCIPAL
    order: 1
    group_id: GROUP_1
```

```yaml
  from_sources: [PRIN_CASH]
  to_targets: [PA, PB, PC, PD, EO]
  payment_style: SEQUENTIAL
  cap_mode: PLANNED
  description: "Aggregate Group I to its Planned Balance"

# Step 3 — PAC II to its Planned Balance
- rule_id: r_prin_pac_ii_planned
  rule_type: PAY_PRINCIPAL
  order: 2
  group_id: GROUP_1
  from_sources: [PRIN_CASH]
  to_targets: [TA, TB]
  payment_style: SEQUENTIAL
  cap_mode: PLANNED
  description: "Aggregate Group II to its Planned Balance"

# Step 4 — Z-bond
- rule_id: r_prin_Z
  rule_type: PAY_PRINCIPAL
  order: 3
  group_id: GROUP_1
  from_sources: [PRIN_CASH]
  to_targets: [Z]
  payment_style: SEQUENTIAL
  description: "Z to zero"

# Step 5a — 95.65 / 4.35 face-weighted split
- rule_id: r_supp_split
  rule_type: SPLIT_CASH
  order: 4
  group_id: GROUP_1
  from_sources: [PRIN_CASH]
  to_targets: [WAWG_BUCKET, PO_BUCKET]
  target_weights: [0.956521694276, 0.043478305724]
  description: "Face-weighted split of remaining principal:
      95.65% to WA-WG, 4.35% to PO"

# Step 5b — WA-WG cascade (multi-target sequential)
- rule_id: r_pay_wawg
  rule_type: PAY_PRINCIPAL
  order: 5
  group_id: GROUP_1
  from_sources: [WAWG_BUCKET]
  to_targets: [WA, WB, WC, WD, WE, WG]
  payment_style: SEQUENTIAL

# Step 5c — Support PO
```

```yaml
- rule_id: r_prin_PO
  rule_type: PAY_PRINCIPAL
  order: 6
  group_id: GROUP_1
  from_sources: [PO_BUCKET]
  to_targets: [PO]

# Step 5d — Sweep leftover bucket cash back into PRIN_CASH
- rule_id: r_supp_sweep_back
  rule_type: SPLIT_CASH
  order: 7
  group_id: GROUP_1
  from_sources: [WAWG_BUCKET, PO_BUCKET]
  to_targets: [PRIN_CASH]
  target_weights: [1.0]

# Step 6 — PAC II cleanup
- rule_id: r_prin_pac_ii_uncapped
  rule_type: PAY_PRINCIPAL
  order: 8
  group_id: GROUP_1
  from_sources: [PRIN_CASH]
  to_targets: [TA, TB]
  payment_style: SEQUENTIAL
  cap_mode: NONE
  description: "Aggregate Group II to zero, without regard to
      Planned Balance"

# Step 7 — PAC I cleanup
- rule_id: r_prin_pac_i_uncapped
  rule_type: PAY_PRINCIPAL
  order: 9
  group_id: GROUP_1
  from_sources: [PRIN_CASH]
  to_targets: [PA, PB, PC, PD, EO]
  payment_style: SEQUENTIAL
  cap_mode: NONE

# Step 7b — Support cleanup (in case face-weighted split left
      tail residue)
- rule_id: r_prin_supports_uncapped
  rule_type: PAY_PRINCIPAL
  order: 10
  group_id: GROUP_1
  from_sources: [PRIN_CASH]
  to_targets: [WA, WB, WC, WD, WE, WG, PO]
  payment_style: SEQUENTIAL
  cap_mode: NONE
```

```yaml
# Step 8 — Residual
- rule_id: r_resid_int
  rule_type: PAY_RESIDUAL
  order: 11
  group_id: GROUP_1
  from_sources: [INT_CASH]
  to_targets: [R]
  payment_style: SEQUENTIAL

- rule_id: r_resid_prin
  rule_type: PAY_RESIDUAL
  order: 12
  group_id: GROUP_1
  from_sources: [PRIN_CASH]
  to_targets: [R]
  payment_style: SEQUENTIAL
```

**Rule count: 13** (one per logical waterfall step), versus the current FNR fixture's **35 rules** (one per bond per step). The behavior is identical — the runtime walks `to_targets` sequentially with the right cap_mode — but the IR is now readable as prospectus paragraphs.

---

## Reusability principles

1. **One bond schema covers all bond types.** The `BondDef` shape accommodates PAC, TAC, Z, IO, PO, sequential, support, residual, pseudo via the `tranche_type` + `tranche_behavior` + schedule / accrual / tracking fields. **Don't add a** `PACBondDef` **or** `ZBondDef` **subclass** — encode the variation in the existing fields.

2. **One rule schema covers all rule types.** Most prospectus structural variation is captured by the *combination* of `rule_type` + `payment_style` + `cap_mode` + `condition_trigger`. PAC / TAC / cleanup / mezz cascades / interest waterfalls / principal waterfalls / fee priorities all fit. **Don't add** `PAY_PRINCIPAL_PAC_SCHEDULE` **or similar specialized rule types** — encode the variation in `cap_mode` and the bond's `tranche_behavior`.

3. **One rule per prospectus step, not one per bond.** The prospectus says "Pay sequentially to PA, PB, PC, PD, EO until their planned balances." That is **ONE rule** with `to_targets: [PA, PB, PC, PD, EO]` and `cap_mode: PLANNED`. Do NOT fragment it into 5 rules.

4. **Built-in tokens are reused everywhere.** `CASH` / `INT_CASH` / `PRIN_CASH` / `LOSS` are universal. Multi-group deals add `GROUP_<id>_*` prefixes which the runtime auto-resolves when `group_id` is set on the rule.

5. **Asset-class differences are parametric, not structural.** Prime auto vs subprime auto: same grammar, more classes / more named amounts / lower fee cap. Agency MBS vs non-agency RMBS: different feature mix (PAC vs OC/stepdown), but the same primitives.

# Human-readability principles

1. **Stable, descriptive `rule_id`s that match the prospectus.** `r_prin_pac_i_planned` reads as "rule, principal, PAC I, to planned balance" — a person can scan the IR and match it to the prospectus paragraph. Avoid synthetic ids like `r_principal_0001` that lose intent.

2. `description` **field on every rule.** One sentence, ideally verbatim from the prospectus. The IR should be readable on its own without referring back to the source document.

3. **Group IR top-level lists by purpose, not alphabetically.** `bonds` ordered by seniority; `waterfall_rules` ordered by priority (matching `order`); `triggers` ordered by deal-state relevance. The IR document reads top-to-bottom as the prospectus reads.

4. **Comments allowed via `description` even where Pydantic doesn't normally permit.** Every node has a `description: str` field. Use it. Especially for:

    - Rules: paste the prospectus phrasing verbatim.
    - Bonds: note the rationale for any non-obvious field (e.g., why `support_tranches` is set as it is).
    - Triggers: explain the threshold table and cure logic.

5. **Compact over verbose where math is identical.** If a rule could be written as one multi-target rule or as N single-target rules, prefer the compact form because it matches the prospectus. The runtime produces identical output either way; the compact form is far easier to read and edit.

6. **Avoid hidden runtime knobs.** If a deal needs an exotic behavior (Z accrual, schedule cap mode, face-weighted split), make it visible in the IR via existing fields (`cap_mode`, `target_weights`, `pay_mode=PIK`). Do not bury it in `deal_knobs` or in code branches in the runtime.

7. **One DealDefinition file per real-world deal.** A single, self-contained, version-controlled JSON / YAML / Python factory. The fixture for FNR 2006-018 is one file: `tests/fixtures/fnr_2006_018/deal_definition.py`. The full combined Group 1 + Group 2 deal is one `DealDefinition` with multiple `collateral_groups`, not two separate files.

# Anti-patterns to avoid

| Anti-pattern | Better approach |
|---|---|
| One rule per bond ("`r_int_PA`, `r_int_PB`, …") | One rule per waterfall step (`r_int_cascade` with all bonds) |
| New `RuleType` for each prospectus phrase | Existing rule types + `cap_mode` + `payment_style` |
| Subclassing `BondDef` for PAC / Z / IO | Set `tranche_type` + `tranche_behavior` + `schedule_contract` / `tracks_bonds` |
| Ad-hoc behavioral key in `deal_knobs` (not one of the 5 reserved keys) | Visible IR fields (`target_weights`, `cap_mode`, `pay_mode`) — promote new behavior to a real schema field |
| Synthetic `rule_id` like `r_001` | Descriptive `r_prin_pac_i_planned` |

| Anti-pattern | Better approach |
|---|---|
| Empty `description` field | Verbatim prospectus phrasing |
| Two `DealDefinition` files for one prospectus deal with multiple groups | One DealDefinition with multiple `collateral_groups` |
| Trigger logic inline in expressions | Named `TriggerNode` referenced by `condition_trigger` |
| Computed amount inline in `max_amount_expr` | Named `CalculationNode` (or proposed `ComputedAmountNode`) |